UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN
PLAN FIN DE SEMANA

Medicentro - MANUAL TÉCNICO

DYLAN AARON PEÑATE PÉREZ
JOSÉ ARTURO CASTILLO GÁLVEZ
FERNANDO JOSÉ GUERRA TOBAR
ASTRID MELISSA ALVAREZ MÉNDEZ
KEVIN ALEJANDRO REYES REVOLORIO

Guatemala, 21 de noviembre de 2,026

Manual Técnico

Este manual técnico describe la arquitectura, las tecnologías, el modelo de datos, la API y el procedimiento de instalación y despliegue del sistema MediCentro, desarrollado para el Sanatorio Medicentro Barberena, S. A. Está dirigido a desarrolladores y personal técnico que deban dar mantenimiento al sistema, extenderlo o desplegarlo en un nuevo ambiente.

> **Nota:** Este documento complementa a los manuales de usuario de cada módulo (Recepción, Seguros Médicos, Expedientes Clínicos, Laboratorio y Diagnóstico, Emergencias, Hospitalización, Farmacia, Facturación y Cobros, Mantenimiento, y Reportes y Estadísticas), que explican el uso funcional del sistema desde la perspectiva del usuario final.

1. Arquitectura general del sistema

MediCentro sigue una arquitectura cliente-servidor de tres capas, con un frontend web (SPA con renderizado híbrido), una API REST como capa de negocio, y un motor de base de datos relacional:

- Cliente (frontend): aplicación Angular que corre en el navegador del usuario y consume la API mediante HTTP/JSON.
- Servidor de aplicación (backend): API REST en ASP.NET Core que expone los recursos del sistema, valida permisos y aplica las reglas de negocio.
- Base de datos: SQL Server, con el esquema de la base MEDICENTRO accedido vía Entity Framework Core (ORM).

El frontend y el backend son dos proyectos independientes (repositorios separados) que se comunican únicamente por HTTP: el frontend no accede a la base de datos directamente, y toda regla de negocio o validación de permisos vive en el backend, nunca solo en el cliente.

> **Nota:** En desarrollo, el backend corre en https://localhost:7086 y el frontend en http://localhost:4200. La URL del backend que usa el frontend está centralizada en src/environments/environment.ts (desarrollo) y environment.production.ts (producción) — antes de este cambio estaba repetida en 48 archivos de servicio distintos.

2. Tecnologías y versiones

2.1 Frontend

Componente	Tecnología / versión
Framework	Angular 22 (standalone components, signals)
Renderizado	Angular SSR (@angular/ssr) con hidratación en el cliente

Componente	Tecnología / versión
Componentes UI	Angular Material 22 + Angular CDK
Lenguaje	TypeScript
Cliente HTTP	HttpClient de @angular/common/http, con interceptor de autenticación
Programación reactiva	RxJS 7
Generación de documentos	docx (generación de archivos Word desde el propio sistema)
Servidor Node (SSR/producción)	Express

2.2 Backend

Componente	Tecnología / versión
Framework	ASP.NET Core 8 (.NET 8, Web API)
ORM	Entity Framework Core 8, proveedor SQL Server
Autenticación	JWT (JSON Web Token) vía Microsoft.AspNetCore.Authentication.JwtBearer
Hash de contraseñas	BCrypt.Net-Next
Documentación de API	Swagger / Swashbuckle.AspNetCore (solo en ambiente de desarrollo)

2.3 Base de datos

Microsoft SQL Server, base de datos MEDICENTRO.

3. Estructura del proyecto

3.1 Frontend (sistema_medicentro)

El código fuente vive en src/app, organizado por módulo funcional: cada carpeta agrupa los componentes standalone de ese módulo (lista, formulario, diálogos). Carpetas principales:

Carpeta	Contenido
recepcion / pacientes	Registro y gestión de pacientes
consultas-externas	Agenda de citas y atención de consultas
expedientes	Expediente clínico unificado por paciente

Carpeta	Contenido
polizas	Pólizas y aseguradoras
laboratorio	Órdenes y resultados de exámenes
emergencias	Casos de emergencia y compromisos de pago
hospitalizacion	Ingresos, camas, órdenes médicas y notas de enfermería
farmacia	Inventario de medicamentos y recetas
facturacion	Facturas (SAT / Digefact), pagos y cuentas por cobrar
mantenimiento	Catálogos del sistema, empleados, usuarios, roles, permisos, bitácora, papelera
reportes	Indicadores clínicos y financieros
service	Servicios Angular (uno por recurso de la API): llamadas HTTP, caché de catálogos, signals de estado
shared	Componentes y utilidades compartidas: menú, notificaciones (toasts), confirmaciones
interceptors	Interceptor HTTP de autenticación (adjunta el JWT a cada petición)
models	Interfaces TypeScript de las entidades del dominio

Los servicios (carpeta service/) son singletons (providedIn: 'root') que exponen signals de solo lectura con el estado ya cargado (por ejemplo, la lista de hospitalizaciones activas), y métodos Retornar* que devuelven Observables para datos de apoyo (médicos, catálogos). La mayoría de las pantallas de listado no vuelven a pedir los datos al backend en cada render: leen el signal del servicio.

3.2 Backend (MediCentroBackendV3)

Carpeta	Contenido
Controllers/	48 controladores REST, uno por recurso (api/pacientes, api/facturas, api/hospitalizaciones, etc.)

Carpeta	Contenido
Servicios/	Lógica de negocio: una interfaz + implementación por controlador (ej. IPacientesService / PacientesService)
Data/	MedicentroDbContext (EF Core) — 71 DbSet, uno por tabla del dominio
Entidades/	Clases POCO que mapean 1 a 1 con las tablas de la base de datos
Dto/	Objetos de entrada/salida de la API (request/response), separados de las entidades
Seguridad/	JWT (JwtService), hash de contraseñas (PasswordHasher), y el filtro de autorización por módulo (AutorizacionPorRolFilter, MatrizAcceso, AccesoUsuario)
Excepciones/	Excepciones de dominio (ej. RecursoNoEncontradoException) que los controladores traducen a códigos HTTP

El patrón se repite en los 48 recursos: Controller (recibe la petición HTTP, sin lógica de negocio) → IServicio/Servicio (lógica de negocio y acceso a datos vía DbContext) → ApiResponse<T> (envoltorio de respuesta uniforme).

4. Modelo de datos

La base de datos MEDICENTRO tiene 71 tablas (una por cada DbSet del MedicentroDbContext), agrupadas en los módulos funcionales del sistema: Fundamentos (empleados, usuarios, roles, permisos), Pacientes, Citas, Tratamientos y recetas, Expediente clínico (fichas, notas de evolución y enfermería, signos vitales, vacunas, constancias médicas), Laboratorio, Hospitalización (habitaciones, camas, órdenes médicas), Emergencias (casos, compromisos de pago), Seguros y convenios (aseguradoras, pólizas, convenios), Facturación y cobros (facturas SAT/Digefact, pagos, cuentas por cobrar, notas de crédito), Farmacia (inventario, movimientos, alertas de stock), Reportes (vistas de indicadores) y Seguridad/auditoría.

4.1 Motor de catálogos genérico

La mayoría de los campos de tipo/estado/categoría (estado de cita, forma de pago, tipo de examen, etc.) no son tablas de catálogo independientes: son filas de dos tablas genéricas, cat_tipo_catalogo (el catálogo en sí, ej. "ESTADO_CITA") y cat_valor_catalogo (sus valores posibles, ej. "Programada", "Confirmada"). Esto es lo que administra la pantalla "Catálogos del sistema" de Mantenimiento, y lo que exponen los endpoints GET /api/catalogos y GET /api/catalogos/{codigoTipo}.

4.2 Patrón estándar de auditoría

Casi todas las tablas de negocio comparten las mismas columnas de auditoría: fecha_creacion, fecha_modificacion, id_usuario_creacion, id_usuario_modificacion, y una baja lógica mediante la columna activo (bit) en vez de borrado físico. El borrado físico de un registro con DELETE está bloqueado por un trigger generado por tabla; por eso las operaciones "Eliminar" del sistema son, casi siempre, una actualización de activo = 0, y cada módulo tiene una pantalla o pestaña de "Eliminados" para consultarlos o reactivarlos.

4.3 Cambios de esquema

El esquema de la base de datos no se versiona con migraciones de Entity Framework: vive como scripts T-SQL independientes en la carpeta scripts/ del repositorio del frontend. 00_crear_base_datos_desde_cero.sql crea la base completa desde cero, y 00_instalacion_completa.sql es un script de instalación consolidado; el resto de los scripts (agregar_*, corregir_*, sembrar_*) son cambios incrementales aplicados en orden sobre una base ya existente. Ver la sección 6 (Instalación y despliegue) para el orden de ejecución.

5. API y autenticación

5.1 Convención de la API

Todos los endpoints viven bajo el prefijo /api/ (ej. /api/pacientes, /api/hospitalizaciones, /api/facturas), uno por recurso, siguiendo las convenciones REST estándar (GET para consultar, POST para crear, PUT para editar, DELETE para la baja lógica). Toda respuesta, exitosa o no, se envuelve en el mismo objeto ApiResponse<T>:

Campo	Descripción
exito	true si la operación se completó correctamente
mensaje	Mensaje descriptivo del resultado (éxito o error)
datos	El recurso solicitado o creado (null si exito es false)
errores	Lista de mensajes de validación, cuando aplica (ej. campos obligatorios faltantes)

> **Nota:** El frontend nunca asume que una respuesta HTTP 200 significa éxito funcional sin revisar exito: algunas reglas de negocio responden 200 con exito: false y un mensaje explicando por qué no se pudo completar la operación.

5.2 Autenticación (JWT)

El inicio de sesión (POST /api/auth/login, sin autenticación previa) recibe usuario, código de usuario y contraseña, valida la contraseña contra el hash guardado (BCrypt)

y, si es válida, genera un token JWT firmado (HMAC-SHA256) con una expiración configurable (Jwt:ExpiracionMinutos en appsettings.json). El token incluye como claims el id de usuario, nombre de usuario, correo y los roles activos del usuario. El frontend guarda este token en localStorage y lo adjunta a cada petición como encabezado Authorization: Bearer <token> mediante un interceptor HTTP (authInterceptor); si el backend responde 401, el interceptor limpia la sesión y redirige al login.

5.3 Autorización por módulo

Además de exigir un token válido ([Authorize] estándar de ASP.NET), cada petición pasa por un filtro de autorización propio (AutorizacionPorRolFilter) que decide si el usuario puede llamar ese endpoint según los MÓDULOS a los que tiene acceso, no según el nombre literal de su rol. El acceso del usuario (AccesoUsuario) se recalcula en cada petición leyendo de la base de datos la cadena usuario → roles activos → permisos activos de esos roles → módulo de cada permiso, en vez de leerse del token — así un cambio de permisos en Mantenimiento aplica de inmediato, sin que el usuario tenga que volver a iniciar sesión. Un usuario con el rol ADMIN tiene acceso total; cualquier otro usuario solo puede operar sobre los módulos de sus permisos (MatrizAcceso.Permite), y los endpoints de solo lectura (GET) pueden tener una regla más permisiva que los de escritura.

El mismo cálculo de acceso (AccesoUsuario) lo usa el endpoint GET /api/mi-acceso para decirle al frontend qué secciones del menú lateral mostrarle, así el backend y el menú del frontend aplican exactamente la misma regla de permisos.

5.4 CORS

El backend solo acepta peticiones del origen configurado en la política PoliticasCorsPredeterminadas (http://localhost:4200 en desarrollo); al desplegar a un servidor real, este origen debe actualizarse al dominio real del frontend en Program.cs.

6. Instalación y despliegue

6.1 Requisitos previos

- SQL Server (motor de base de datos) con acceso para crear la base MEDICENTRO.
- .NET 8 SDK, para compilar y ejecutar el backend (MediCentroBackendV3).
- Node.js y Angular CLI 22, para compilar y ejecutar el frontend (sistema_medicentro).

6.2 Base de datos

1. Ejecutar scripts/00_crear_base_datos_desde_cero.sql sobre el servidor SQL Server para crear la base MEDICENTRO y su esquema completo.

2. Ejecutar scripts/00_instalacion_completa.sql (instalación consolidada) y, si corresponde a la versión del sistema que se está instalando, los scripts incrementales agregar_*.sql, corregir_*.sql y sembrar_*.sql de la carpeta scripts/, en el orden en que fueron agregados al repositorio.

3. Verificar que exista al menos un usuario con el rol ADMIN para poder iniciar sesión y administrar el resto del sistema desde Mantenimiento (los scripts insertar_usuario.sql y crear_rol_admin_permisos.sql sirven de referencia).

6.3 Backend (MediCentroBackendV3)

1. Configurar la cadena de conexión ConnectionStrings:MedicentroDb en appsettings.json (o appsettings.Development.json) apuntando al servidor SQL Server del paso anterior.

2. Configurar la llave de firma del JWT: en desarrollo, Jwt:Key va en appsettings.Development.json (mínimo 32 caracteres); en un servidor real debe configurarse como variable de entorno Jwt__Key, nunca con una llave que empiece con "DEV-ONLY-" fuera de desarrollo (el sistema rechaza arrancar si detecta esa condición).

3. Ejecutar dotnet run (o dotnet run --launch-profile https en desarrollo) desde la carpeta del proyecto. La API queda disponible en la URL configurada (https://localhost:7086 en desarrollo); con el ambiente en Development, también se expone la documentación interactiva de Swagger.

6.4 Frontend (sistema_medicentro)

1. Ejecutar npm install para instalar las dependencias.

2. Configurar la URL del backend: apiUrl en src/environments/environment.ts (desarrollo) y en environment.production.ts (producción) — esta es la ÚNICA configuración a cambiar al apuntar a un backend distinto de localhost.

3. Para desarrollo: npm run start (equivalente a ng serve), disponible en http://localhost:4200 con recarga automática.

4. Para producción: npm run build genera los artefactos en dist/ (incluye el bundle del navegador y el del servidor SSR); el servidor se levanta con npm run serve:ssr:sistema_medicentro (node dist/sistema_medicentro/server/server.mjs).

6.5 Consideraciones de despliegue a un servidor real

• Actualizar apiUrl en environment.production.ts a la URL pública real del backend (no "localhost").

• Actualizar el origen permitido en la política de CORS del backend (Program.cs) al dominio real del frontend.

• Configurar Jwt__Key como variable de entorno del servidor, no en el archivo de configuración del repositorio.

• Servir el frontend por HTTPS: el navegador bloquea el envío del JWT y otras funciones si el frontend corre en HTTP contra un backend en HTTPS en un dominio distinto.

Resumen

MediCentro es un sistema cliente-servidor con un frontend Angular (SSR) y una API REST en ASP.NET Core sobre SQL Server, con autenticación por JWT y autorización

por módulo calculada en cada petición desde la base de datos. El esquema de 71 tablas se instala y actualiza mediante scripts T-SQL versionados en el propio repositorio, y toda la configuración que cambia entre ambientes (URL del backend, cadena de conexión, llave JWT, origen CORS) está centralizada en un puñado de archivos, para que levantar un ambiente nuevo (desarrollo, pruebas o producción) no requiera tocar el código.